Data Structures

Lecture 15: Linked List

Dr. Shabnam Samima

Content

- **Insertion**

Overflow and Underflow

Overflow

Overflow happens at the time of insertion. If we have to insert new space into the data structure, but there is no free space i.e. availability list is empty, then this situation is called 'Overflow'. The programmer can handle this situation by printing the message of OVERFLOW. Overflow will occur with our linked lists when AVAIL = NULL

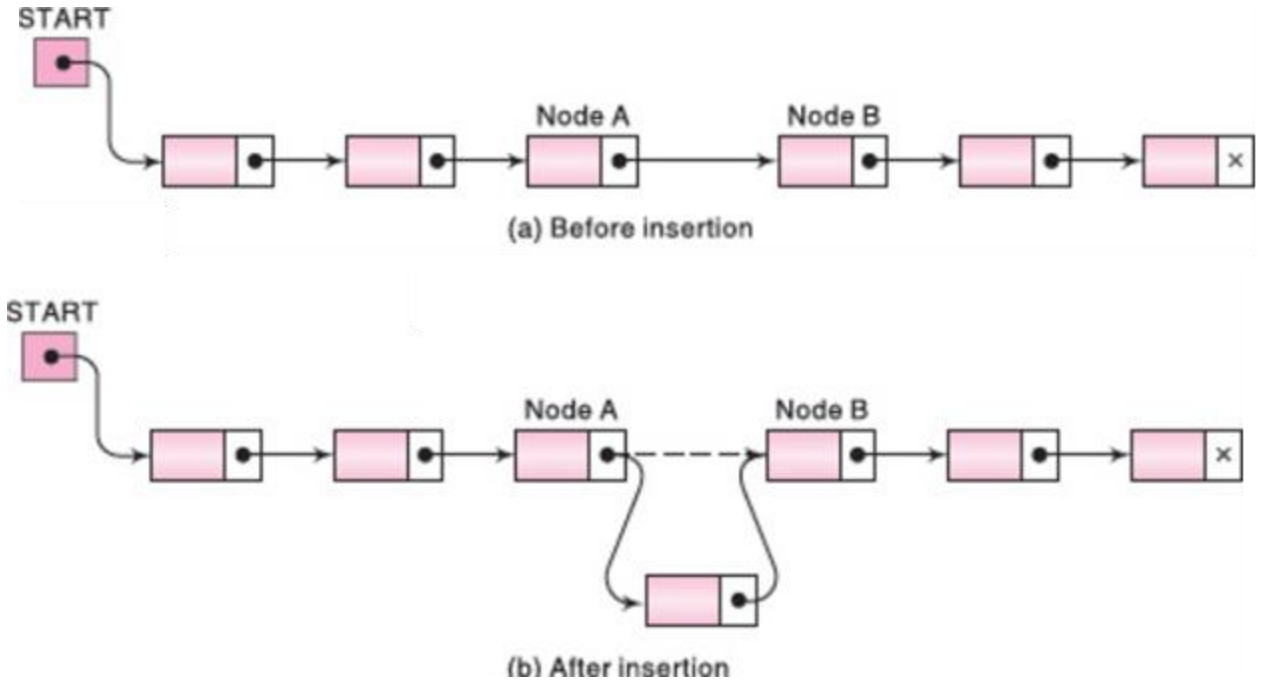
Underflow

Underflow happens at the time of deletion. If we have to delete data from the data structure, but there is no data in the data structure i.e. data structure is empty, then this situation is called 'Underflow'. The programmer can handle this situation by printing the message of UNDERFLOW. Underflow will occur with our linked lists when START = NULL

Linked List: Insert Operation

Insertion in a linked list involves adding a new node at a specified position in the list. Let LIST be a linked list with successive nodes A and B.

- Suppose a node *N* is to be inserted into the list between nodes A and B.
- Node A now points to the new node N,
- Node N points to node B, to which A previously pointed.



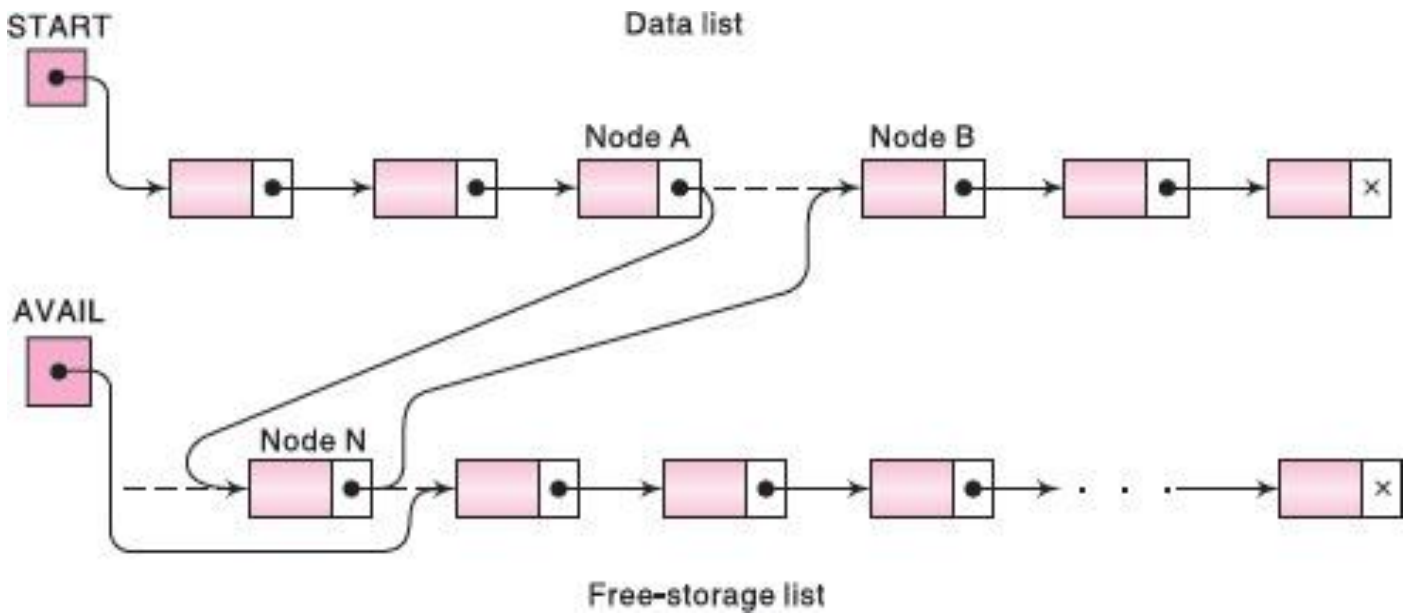
Linked List: Insert Operation

Suppose our linked list is maintained in memory in the form **LIST(INFO, LINK, START, AVAIL)**

The ***next*** pointer field of node ***A*** now points to the new ***node N,*** to which AVAIL previously pointed.

AVAIL now points to the second node in the free pool, to which ***node N*** previously pointed.

The ***next*** pointer field of node N now points to ***node B***, to which ***node A*** previously pointed.



Linked List: Insert Operation

Insertion in a linked list involves adding a new node at a specified position in the list. There are several types of insertion based on the position where the new node is to be added:

- At the front of the linked list
- After a given node.
- Before a given node.
- Between two nodes.
- At the end of the linked list.

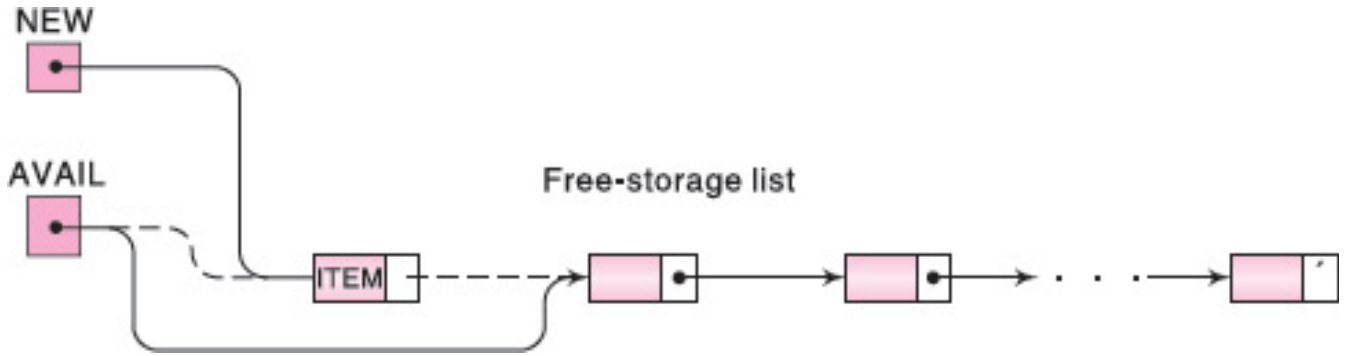
Linked List: Insert Operation

Common Step: Any of the case mentioned in previous slide will consider to insert data in linked list by using a node in the AVAIL list, So the common step required for different cases are as follows

(a) Checking to see if space is available in the AVAIL list. If not, that is, if AVAIL = NULL, then the algorithm will print the message OVERFLOW.

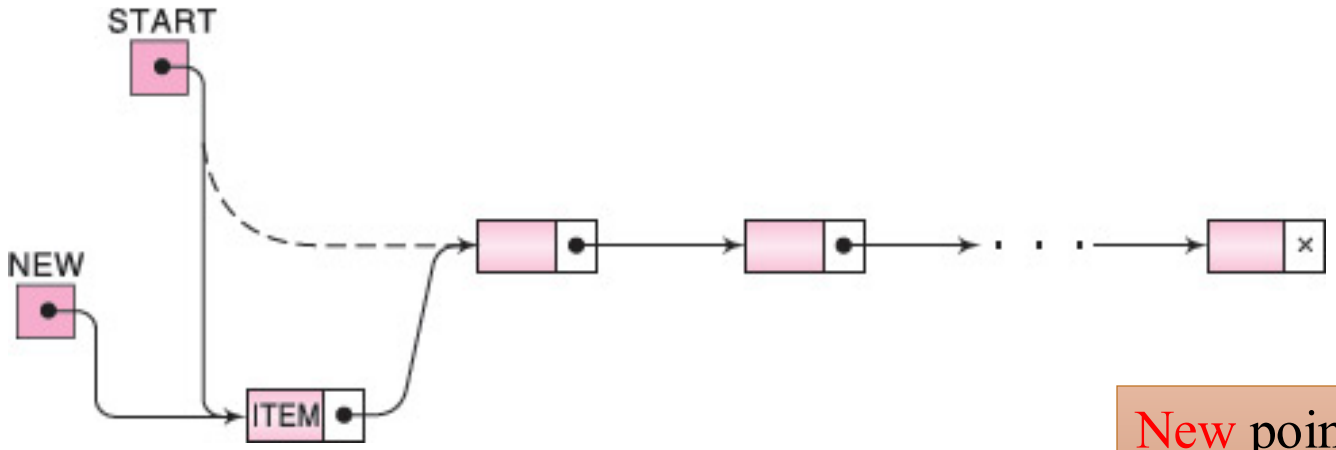
(b) Removing the first node from the AVAIL list. Using the variable NEW to keep track of the location of the new node NEW := AVAIL, AVAIL := LINK[AVAIL]

(c) Copying new information into the new node INFO[NEW] := ITEM



Linked List: Insert at beginning

Suppose our linked list is randomly arranged and there it is not specified as where to insert the new node in the list. Then the easiest place to insert the node is at the beginning of the list.



New points to the ***new node*** to be inserted,
Start points to the ***first node*** of the linked list

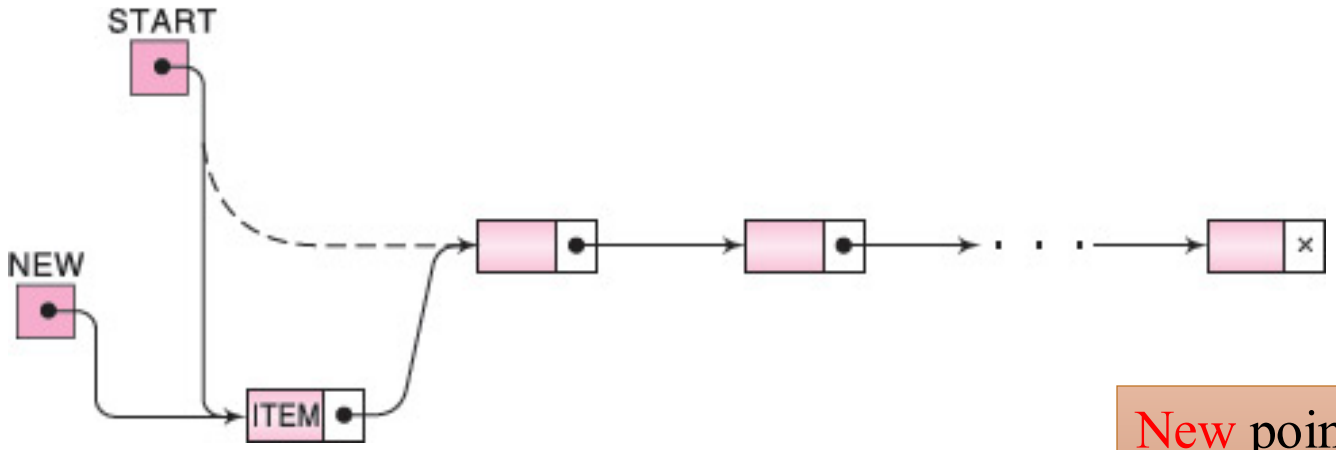
Linked List: Insert at beginning

Algorithm

```
INSFIRST(INFO, LINK, START, AVAIL, ITEM)
            If AVAIL = NULL, then:
                    Write: OVERFLOW, and Exit. //Overflow
            Set NEW := AVAIL and AVAIL := LINK [AVAIL].
                    //Remove first node from AVAIL list.
            Set INFO[NEW] := ITEM.
                    // Copies new data into new node
             Set LINK[NEW] := START.
                    // New node now points to original first node.
            Set START := NEW.
                    // Changes START so it points to the new node.
Exit.
```

Linked List: Inserting after a Given Node

Consider that we are given with the value of LOC where either LOC is the location of a node A in a linked LIST or LOC = NULL. We need to insert new node (say ITEM), so that ITEM follows node A or, when LOC = NULL, that ITEM is the first node.

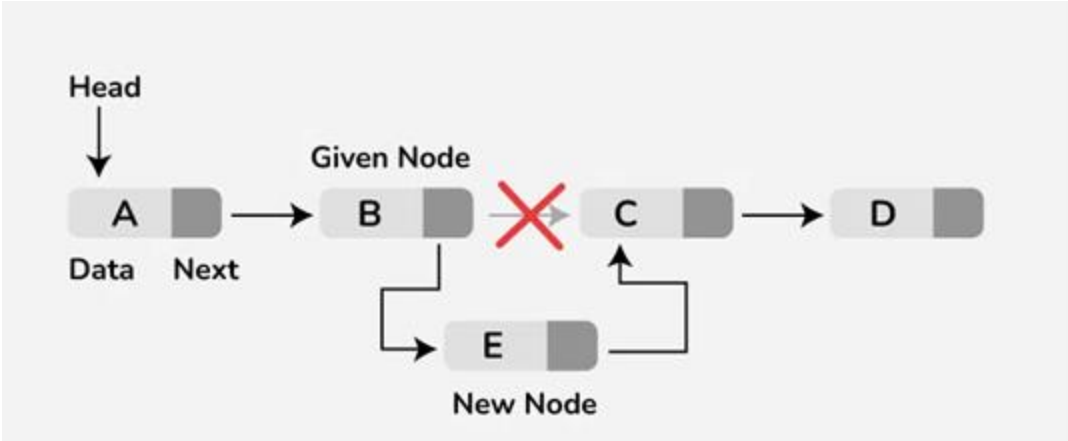


New points to the ***new node*** to be inserted,
Start points to the ***first node*** of the linked list

Linked List: Inserting after a Given Node

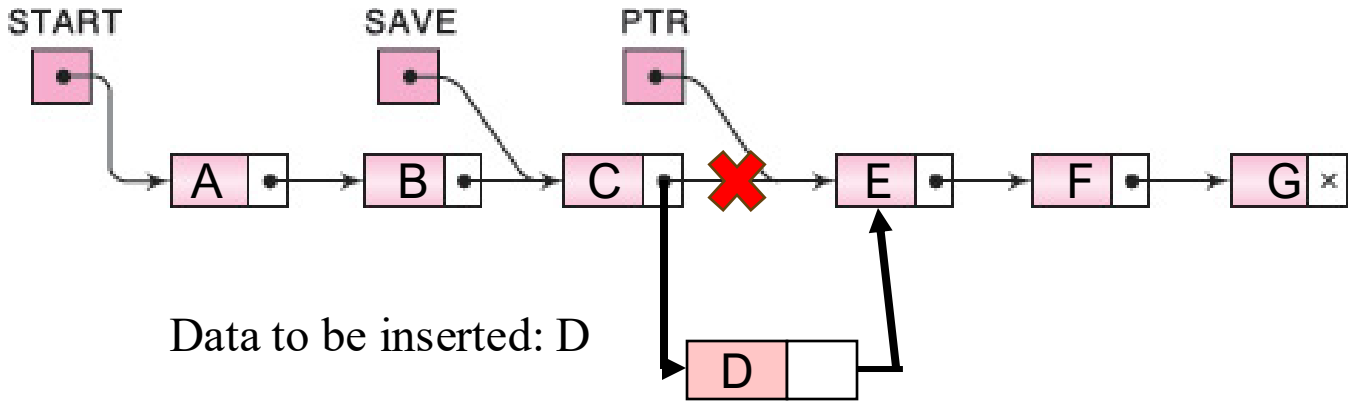
Algorithm

```
INSFIRST(INFO, LINK, START, AVAIL, ITEM)
        If AVAIL = NULL, then:
                Write: OVERFLOW, and Exit. //Overflow
        Set NEW := AVAIL and AVAIL := LINK [AVAIL].
                //Remove first node from AVAIL list.
        Set INFO[NEW] := ITEM.
                // Copies new data into new node
        If LOC:= NULL
                Set LINK[NEW] := START and START=NEW
// New node now points to original first node and Changes START so it points to the new node.
        Else
                Set LINK[NEW] := LINK[LOC] and LINK[LOC] =NEW
        EXIT
```



Linked List: Inserting into a Sorted list

Suppose ITEM is to be inserted into a sorted linked LIST. Then ITEM must be inserted between nodes A and B so that INFO(A) < ITEM ≤ INFO(B)



This requires two steps:

1. Finding correct position in the linked list
2. Update list with new data

Linked List: Inserting after a Given Node

Algorithm

```
FINDA(INFO, LINK, START, ITEM, LOC)
1 If START = NULL // List is empty
      then: Set LOC := NULL, and Return.
2. If ITEM < INFO[START],
      then: Set LOC := NULL, and Return.
3. Set SAVE := START and PTR := LINK[START] //initialize pointers
4. Repeat Steps 5 and 6 while PTR ≠ NULL.
5.    If ITEM < INFO[PTR], then:
            Set LOC := SAVE, and Return.
6.    Otherwise, Set SAVE := PTR and PTR := LINK[PTR].
7.    Set LOC := SAVE.
8. Return.
```

Once the LOC is known use the algorithm ***Inserting after a Given Node*** to insert the data in linked list